Construire une requête SQL

Scénario · transcription complète · Guide-Texte synchronisé

Source	exercices/sql/CommentConstruireunerequêteSQL.html
Production	Full HD · narration française · 10 chapitres
Accessibilité	Guide-Texte incrusté + sous-titres SRT

Scénario pédagogique

La vidéo suit la logique de la page mise à jour : elle part de l’énoncé, distingue les cinq questions de compréhension des sept étapes de construction, montre les clauses SQL dans l’ordre, puis traite les sous-requêtes et les auto-jointures. Chaque chapitre associe une zone de raisonnement, une zone SQL et un bandeau Guide-Texte synchronisé avec la narration.

Découpage

01. Introduction
02. Les questions et les sept étapes
03. Étape 1 — SELECT
04. Étape 2 — FROM et tables
05. Étape 3 — Relier les tables
06. Étape 4 — WHERE et sous-requête
07. Étapes 5 et 6 — GROUP BY et HAVING
08. Étape 7 — ORDER BY et contrôles
09. Cas particulier — Auto-jointure
10. Assemblage complet et synthèse

Chapitre 01 — Introduction

Guide-Texte synchronisé

01	Page mise à jour : 7 étapes + contrôles automatiques
02	Nouveautés : sous-requête et auto-jointure
03	Partir de l’énoncé — jamais du code au hasard
04	Guide-Texte : l’idée essentielle reste visible
05	Objectif : construire, contrôler, puis tester

Transcription complète

Bienvenue dans S T I version deux point zéro. Dans cette vidéo, on va détailler la démarche à faire pour créer une requête SQL. Nous nous appuyons sur la page comment créer une requête SQL SELECT. Cette page interactive mise à jour intègre maintenant sept étapes, les contrôles automatiques, les sous-requêtes et l’auto-jointure. Le principe reste simple : nous ne commençons pas par écrire du code au hasard. Nous partons de l’énoncé, nous repérons les mots importants, puis nous répondons à chaque question dans l’ordre. À l’écran, le Guide-Texte affiche en permanence l’idée essentielle de l’explication en cours. À la fin, vous saurez transformer un énoncé en clauses SELECT, FROM, WHERE, GROUP BY, HAVING et ORDER BY, puis vérifier la cohérence de la requête avant de la tester.

Chapitre 02 — Les questions et les sept étapes

Guide-Texte synchronisé

01	5 questions pour comprendre l’énoncé
02	7 étapes pour construire la requête
03	Tables et jointures sont traitées séparément
04	Calcul avant comparaison → sous-requête
05	Même table deux fois → auto-jointure

Transcription complète

Avant toute saisie, découpons l’énoncé. Les cinq questions principales sont : que veut-on afficher ? Dans quelles tables se trouvent les informations ? Faut-il filtrer des lignes ? Faut-il calculer par groupe ? Et faut-il filtrer le résultat du calcul ? La page transforme ce raisonnement en sept étapes opérationnelles. Elle sépare le choix des tables de leur liaison, puis ajoute le tri final. Nous obtenons donc : afficher, choisir les tables, relier les tables, filtrer les lignes, calculer par groupe, filtrer les groupes, et trier. Deux tests complètent cette méthode. Si un calcul doit être effectué avant une comparaison, pensez à une sous-requête. Si la même table intervient deux fois, pensez à une auto-jointure avec deux alias différents.

Chapitre 03 — Étape 1 — SELECT

Guide-Texte synchronisé

01	« afficher », « lister », « combien » → SELECT
02	Calculs : COUNT · SUM · AVG · MIN · MAX
03	AS donne un nom clair au résultat calculé
04	Sans doublons → DISTINCT
05	SELECT décrit uniquement le résultat attendu

Transcription complète

Étape un : que veut-on afficher ? Repérez les verbes afficher, donner, lister, quel est ou combien. Une colonne demandée va dans SELECT. Un calcul demandé utilise une fonction d'agrégation : COUNT pour compter, SUM pour additionner, AVG pour une moyenne, MIN pour le minimum et MAX pour le maximum. Donnez un alias clair au calcul, par exemple moyenne. Si l'énoncé demande une liste sans doublons, activez DISTINCT. Dans notre exemple, afficher le libellé de chaque classe et la moyenne des notes donne SELECT classe point libelle, virgule AVG de note point valeur, AS moyenne. À ce stade, nous avons défini le résultat attendu, pas encore les tables ni les conditions.

Chapitre 04 — Étape 2 — FROM et tables

Guide-Texte synchronisé

01	FROM contient toutes les tables utiles
02	Ajouter une table pont si la liaison l’exige
03	classe → eleve → note
04	Même table plusieurs fois → alias distincts
05	Un alias identifie chaque exemplaire sans ambiguïté

Transcription complète

Étape deux : dans quelles tables se trouvent ces informations ? Les colonnes sélectionnées permettent à la page de cocher automatiquement leurs tables. Ajoutez aussi une table pont lorsqu’elle est nécessaire pour relier les autres. Dans notre base lycée, le libellé se trouve dans classe et la valeur dans note. Mais classe et note ne sont pas directement reliées : la table eleve sert de pont. FROM contient donc classe, eleve et note. Lorsque la même table doit intervenir plusieurs fois, la page permet d’ajouter une copie et impose des alias distincts, par exemple eleve e1 et eleve e2. Un alias n’est pas décoratif : il identifie précisément chaque exemplaire et évite les colonnes ambiguës.

Chapitre 05 — Étape 3 — Relier les tables

Guide-Texte synchronisé

01	Jointure : clé primaire = clé étrangère
02	Les conditions de liaison sont écrites dans WHERE
03	3 tables en chaîne → 2 jointures
04	Règle de contrôle : T tables → T − 1 liens
05	Lien manquant → produit cartésien

Transcription complète

Étape trois : comment relier les tables ? Dans cette méthode, les jointures sont écrites dans WHERE sous la forme table un point colonne égale table deux point colonne. Chaque lien associe une clé primaire à la clé étrangère correspondante. Avec trois tables reliées en chaîne, il faut normalement deux conditions d’égalité : classe point id égale eleve point id classe, puis eleve point id égale note point id eleve. La page peut suggérer les jointures à partir du schéma, mais vous devez toujours les relire. Retenez le contrôle essentiel : avec T tables, il faut en général T moins un liens d’égalité. Une liaison manquante produit un produit cartésien, donc beaucoup trop de lignes et un résultat faux.

Chapitre 06 — Étape 4 — WHERE et sous-requête

Guide-Texte synchronisé

01	WHERE filtre les lignes avant le regroupement
02	Pas de fonction AVG ou COUNT directement dans WHERE
03	Calcul nécessaire avant comparaison → sous-requête
04	=, >, < attendent une seule valeur
05	IN et NOT IN acceptent une liste

Transcription complète

Étape quatre : faut-il filtrer des lignes ? Les expressions dont, supérieur à, de la classe, en deux mille vingt-cinq ou commence par indiquent souvent un filtre WHERE. WHERE agit avant les regroupements, sur les lignes individuelles. N’écrivez donc jamais AVG, COUNT ou une autre fonction d’agrégation directement sur la colonne du WHERE. Si la valeur de comparaison doit d’abord être calculée, utilisez une sous-requête. Par exemple, pour trouver les élèves dont une note dépasse la moyenne générale, on écrit note point valeur supérieur à, entre parenthèses, SELECT AVG de note point valeur FROM note. Avec égal, supérieur ou inférieur, la sous-requête doit renvoyer une seule valeur. Avec IN ou NOT IN, elle peut renvoyer une liste.

Chapitre 07 — Étapes 5 et 6 — GROUP BY et HAVING

Guide-Texte synchronisé

01	« pour chaque », « par classe » → GROUP BY
02	Colonne affichée sans calcul → présente dans GROUP BY
03	Regrouper par l'identifiant évite les homonymes
04	WHERE filtre avant le calcul
05	HAVING filtre les groupes après le calcul

Transcription complète

Étape cinq : faut-il calculer par groupe ? Les expressions pour chaque, par classe ou selon indiquent GROUP BY lorsqu'un calcul est demandé. Toute colonne affichée sans fonction d'agrégation doit figurer dans le GROUP BY. Pour éviter de fusionner deux classes homonymes, regroupez d'abord par l'identifiant, et ajoutez le libellé si le moteur SQL l'exige. Étape six : faut-il filtrer le résultat du calcul ? Une phrase comme dont la moyenne dépasse douze devient HAVING AVG de note point valeur supérieur à douze. La différence est fondamentale : WHERE filtre les lignes avant le calcul, tandis que HAVING filtre les groupes après le calcul. HAVING peut aussi comparer son agrégat à une sous-requête, par exemple à une moyenne générale calculée séparément.

Chapitre 08 — Étape 7 — ORDER BY et contrôles

Guide-Texte synchronisé

01	ORDER BY organise le résultat final
02	ASC : croissant · DESC : décroissant
03	Trier sur l’alias du calcul est plus lisible
04	Lire toutes les remarques automatiques
05	Structure cohérente ≠ résultat déjà prouvé

Transcription complète

Étape sept : faut-il trier ? Les expressions trié par, classé, du plus petit au plus grand ou du plus grand au plus petit indiquent ORDER BY. ASC correspond à l’ordre croissant et DESC à l’ordre décroissant. Pour notre moyenne, ORDER BY moyenne DESC place la meilleure classe en premier. Avant de copier la requête, lisez les remarques automatiques. La page signale une jointure manquante, une condition incomplète, une colonne du SELECT absente du GROUP BY, un HAVING sans GROUP BY ou une sous-requête incohérente. Le message aucun problème détecté ne remplace pas le test sur la base : il confirme seulement que la structure paraît cohérente. Relisez toujours l’énoncé, puis exécutez la requête sur des données connues.

Chapitre 09 — Cas particulier — Auto-jointure

Guide-Texte synchronisé

01	Auto-jointure : une table joue deux rôles
02	Créer deux exemplaires : eleve e1 et eleve e2
03	Même classe : e1.id_classe = e2.id_classe
04	Éviter la ligne elle-même : e1.id <> e2.id
05	Éviter les paires inversées : e1.id < e2.id

Transcription complète

Voyons le deuxième cas particulier : l’auto-jointure. Elle est nécessaire lorsque la même table joue deux rôles dans la question, par exemple trouver deux élèves appartenant à la même classe. Ajoutez une copie de la table eleve, puis nommez les deux exemplaires e1 et e2. Dans FROM, écrivez eleve e1, eleve e2. Reliez leurs classes avec e1 point id classe égale e2 point id classe. Ajoutez aussi e1 point id différent de e2 point id pour ne pas associer un élève avec lui-même. Selon le résultat attendu, une condition comme e1 point id inférieur à e2 point id évite d’afficher deux fois la même paire dans l’ordre inverse. Le rôle des alias est ici indispensable : sans eux, SQL ne saurait pas quel exemplaire de la table vous désignez.

Chapitre 10 — Assemblage complet et synthèse

Guide-Texte synchronisé

01	SELECT → résultat demandé
02	FROM + WHERE → tables et liaisons
03	GROUP BY + HAVING → calculs par groupe
04	ORDER BY → ordre final
05	Comprendre → construire → contrôler → tester

Transcription complète

Assemblons maintenant la requête complète. Nous voulons, pour chaque classe, afficher son libellé et la moyenne des notes, seulement si cette moyenne dépasse douze, de la plus grande à la plus petite. SELECT contient classe point libelle et AVG de note point valeur AS moyenne. FROM contient classe, eleve et note. WHERE relie classe à eleve, puis eleve à note. GROUP BY utilise classe point id et classe point libelle. HAVING conserve les groupes dont AVG de note point valeur dépasse douze. ORDER BY moyenne DESC termine la requête. La méthode se résume ainsi : comprendre l'énoncé, construire chaque clause dans l'ordre logique, contrôler les jointures et les agrégats, puis tester. Si un calcul précède la comparaison, utilisez une sous-requête. Si une table joue deux rôles, utilisez une auto-jointure. Votre requête devient alors la traduction fidèle d'un raisonnement clair.

Requêtes de référence

Agrégation par classe

```
SELECT classe.libelle, AVG(note.valeur) AS moyenne
FROM classe, eleve, note
WHERE classe.id = eleve.id_classe
  AND eleve.id = note.id_eleve
GROUP BY classe.id, classe.libelle
HAVING AVG(note.valeur) > 12
ORDER BY moyenne DESC;
```

Sous-requête : note supérieure à la moyenne générale

```
SELECT DISTINCT eleve.nom
FROM eleve, note
WHERE eleve.id = note.id_eleve
  AND note.valeur > (
    SELECT AVG(note.valeur)
    FROM note
  );
```

Auto-jointure : deux élèves de la même classe

```
SELECT e1.nom, e2.nom
FROM eleve e1, eleve e2
WHERE e1.id_classe = e2.id_classe
  AND e1.id < e2.id;
```